

Interconnection Structures

Condensed Study Notes

Generated: 2026-05-24 23:46 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

This lecture introduces the fundamental concepts of computer architecture and organization, focusing on how computers are designed and how they work internally.

At its core, a computer can be understood as a system that processes information. This processing is managed by a central component, often referred to as the **CPU (Central Processing Unit)**. The CPU is essentially the 'brain' of the computer, responsible for carrying out the instructions given to it by software programs.

To understand how the CPU works, we can think of it as having a core cycle of operations. This is known as the **fetch-decode-execute cycle**.

- **Fetch:** The CPU retrieves an instruction from the computer's memory. Think of this like the CPU looking up the next step in a recipe book.
- **Decode:** The CPU figures out what the instruction means. It translates the coded instruction into an action it can perform.
- **Execute:** The CPU performs the action specified by the instruction. This could involve calculations, moving data, or making decisions.

This cycle repeats continuously, allowing the computer to run programs.

Another important aspect influencing a computer's speed is its **clock speed**. This is measured in Gigahertz (GHz) and essentially tells us how many of these fetch-decode-execute cycles the CPU can perform in one second. A higher clock speed means the CPU can process instructions faster.

However, it's crucial to remember that a faster CPU doesn't automatically guarantee a faster overall computer. Other components, such as **memory** (where data and instructions are stored) and **I/O (Input/Output)** devices (like keyboards, screens, and hard drives), also play a significant role in a computer's performance.

Interconnection Structures

Computers are built with three essential modules: the Processor (which we've learned about, including its fetch-decode-execute cycle), Memory (which stores data and instructions), and Input/Output (I/O) devices (which allow interaction with the outside world).

For these modules to work together, they need to communicate. The pathways that connect these modules are collectively known as the **interconnection structure**. The design of this structure is driven by the type and frequency of communication required between the different modules.

Think of the interconnection structure like the road system in a city. The Processor, Memory, and I/O devices are like different districts. The roads (interconnection structure) allow traffic (data and control signals) to flow between these districts, enabling them to function as a cohesive whole. Different road

layouts (bus structures) can affect how efficiently traffic moves. The raw content mentions two types:

- **Single bus:** A single pathway for all communication.
- **Multiple bus:** Multiple pathways, potentially allowing for more simultaneous communication.

I/O module

From an internal perspective within the computer system, the I/O (Input/Output) module performs functions that are quite similar to memory.

An I/O module can manage more than one external device. Each connection to an external device is called a **port**, and each port is given a unique address. There are also external pathways for sending data into and out of these devices.

Interconnection structures, which are the pathways for communication between the processor, memory, and I/O modules, need to support several types of data transfers:

- **Processor to Memory:** The processor writes data to memory.
- **Memory to Processor:** The processor reads data or instructions from memory.
- **Processor to I/O:** The processor sends data to an I/O device.
- **I/O to Processor:** The processor reads data from an I/O device.
- **I/O to or from Memory:** In these cases, the I/O module can directly exchange data with memory without involving the processor. This is known as **Direct Memory Access (DMA)**. Think of DMA as a special courier service that allows devices to send packages (data) directly to and from the warehouse (memory) without needing to go through the main office (processor) for every single delivery.

Additionally, the I/O module can receive **interrupt signals** from external devices. These signals act like a notification system, alerting the processor that an I/O device needs attention, such as when a user presses a key or a file has finished downloading.

Memory

Memory is where a computer stores data and instructions. Think of it like a vast collection of mailboxes, each with a unique number.

- **Address:** Each piece of information (called a 'word') in memory has a specific location, identified by a unique numerical address, much like each mailbox has a street number.
- **Operations:** The computer can perform two main operations on memory: it can **read** data from a specific address, or it can **write** data into a specific address. These operations are controlled by signals.
- **Words:** Memory is typically organized into a set number of words (let's say 'N' words), and each word has the same length.

Complex Instruction Set Architecture (CISC)

Complex Instruction Set Architecture (CISC) is an approach to CPU design that aims to increase performance.

The core idea behind CISC is to create hardware that is complex enough to handle very intricate, low-level language statements. In this architecture, a single instruction can perform multiple operations that would typically require several instructions in other architectures.

Think of it like a highly specialized tool. Instead of having separate tools for cutting, shaping, and finishing a piece of wood, a CISC instruction is like a multi-tool that can do all those things with a single command. For example, a single multiplication instruction in CISC might automatically handle loading the data to be multiplied, performing the multiplication, and then storing the result, all in one go. This makes the hardware more complex but can potentially simplify the software needed to perform these tasks.

Instruction set Architecture: CISC, RISC

The Instruction Set Architecture (ISA) defines the set of commands that a CPU can understand and execute. Think of it as the vocabulary a CPU knows. Different ISAs exist, and two major types are CISC and RISC.

Previously, we learned about CISC (Complex Instruction Set Computer) architecture. CISC uses complex hardware to execute sophisticated, multi-step operations with a single instruction. This aims to boost CPU performance by reducing the number of instructions needed for a task.

RISC (Reduced Instruction Set Computer) architecture, in contrast, uses a simpler set of instructions. Each RISC instruction is designed to be very basic and execute quickly. Complex operations are achieved by combining multiple simple RISC instructions. This approach aims for efficiency by making each individual instruction very fast to process.

In essence:

- **CISC:** Many complex instructions, fewer instructions per program.
- **RISC:** Fewer simple instructions, more instructions per program.

Reduced Instruction Set Architecture (RISC)

RISC (Reduced Instruction Set Architecture) is an approach to designing a CPU's instruction set. The core idea behind RISC is to simplify the hardware by using a smaller, more basic set of instructions.

These instructions focus on fundamental operations like:

- **Loading:** Bringing data from memory into the CPU.
- **Evaluating:** Performing calculations or operations on data.
- **Storing:** Writing data back to memory.

Think of it like building with LEGOs: instead of having one super-specialized brick that does many things, RISC uses a set of very simple, fundamental bricks that can be combined in many ways to achieve complex results. This simplicity in the instructions allows for simpler hardware, which can lead to faster execution of each individual instruction.

The “best” way to design a CPU has been a subject of debate

The design of a CPU has sparked ongoing discussion, particularly regarding the approach to its instruction set. This leads us to two main architectural philosophies: CISC and RISC.

- **CISC (Complex Instruction Set Computer):** This approach uses a large set of complex machine language instructions. Each instruction can perform a sophisticated, multi-step operation. Think of it like having a special tool for every single intricate task – one button press does a lot.
- **RISC (Reduced Instruction Set Computer):** In contrast, RISC employs a smaller, simpler set of instructions. These simpler commands execute faster, but it might take more individual instructions to complete a complex task. This is more like having a basic set of versatile tools that you combine to build something complex.

The core debate revolves around whether it's better to have powerful, longer commands that require fewer instructions overall (CISC), or shorter, simpler commands that execute more quickly individually, potentially leading to more efficient code despite needing more instructions (RISC).

Characteristic of RISC

RISC (Reduced Instruction Set Computer) architecture is characterized by several key features that contribute to its design philosophy:

- **Simpler Instructions:** RISC processors use simpler instructions, which makes the process of decoding them (figuring out what they mean) easier and faster.
- **Single-Word Instructions:** Each instruction is designed to fit within a single word of memory. A 'word' is the standard unit of data that a computer's processor can handle at one time.
- **Single Clock Cycle Execution:** Ideally, each instruction takes only one clock cycle to complete. A 'clock cycle' is the basic time unit for the CPU's operations, similar to a heartbeat.
- **More General-Purpose Registers:** RISC designs typically include a larger number of general-purpose registers. Registers are small, very fast storage locations within the CPU itself, used to hold data and instructions currently being worked on. Having more registers means the CPU can keep more data readily available, reducing the need to constantly access slower main memory.
- **Simple Addressing Modes:** 'Addressing modes' refer to the ways in which instructions specify the location of data they need. RISC uses simpler methods for this, making instruction execution more predictable.
- **Fewer Data Types:** RISC architectures support a more limited set of data types compared to CISC. This simplification helps in designing more streamlined hardware.
- **Pipelining Achieved:** These characteristics, particularly the single-cycle execution and simple instructions, make RISC architectures well-suited for 'pipelining'. Pipelining is a technique where the CPU can start working on the next instruction before the current one is fully finished, like an assembly line where different stages of production happen simultaneously.

Characteristic of CISC

CISC (Complex Instruction Set Computer) architecture is characterized by several key features that distinguish it from other designs.

Key characteristics include:

1. **Complex Instructions and Decoding:** CISC instructions are often intricate and require complex hardware to decode their meaning. This means the CPU has to do more work to figure out what a single instruction is asking it to do.
2. **Larger Instruction Size:** CISC instructions can be longer than the standard word size of the computer's memory. Think of it like trying to fit a long sentence onto a small sticky note – it might take up more space than a single word.
3. **Multi-Cycle Execution:** Unlike simpler instructions that can be completed in one clock tick (a single cycle), CISC instructions may require multiple clock cycles to execute. This is because a single CISC instruction can perform a series of operations that a simpler design would break down into several steps.
4. **Fewer General-Purpose Registers:** Because CISC operations can often be performed directly on data stored in main memory, there's less need for a large number of temporary storage locations within the CPU itself, known as registers. The CPU can access and manipulate data right where it's stored in memory.
5. **Complex Addressing Modes:** CISC architectures support a variety of complex ways to specify the location of data (addressing modes). This allows for powerful and flexible data access, but also adds to the complexity of the instruction decoding process.
6. **More Data Types:** CISC designs typically support a wider range of data types directly within their instruction set, simplifying certain programming tasks by allowing operations on different kinds of data with single instructions.

CISC vs. RISC: Which is Better?

The question of whether CISC (Complex Instruction Set Computer) or RISC (Reduced Instruction Set Computer) is the "better" design method doesn't have a single, definitive answer. Both approaches have their strengths and weaknesses.

- **CISC's Goal:** To minimize the number of instructions needed for a program by making each instruction perform more complex tasks. This was particularly useful in the early days of assembly language programming, which was tedious and prone to errors. Think of CISC like using a specialized tool that can do a very specific, multi-step job in one go.
- **RISC's Goal:** To reduce the number of clock cycles required for each instruction, even if it means a program needs more individual instructions. This approach favors simpler, faster individual operations. Imagine RISC as using a set of basic, versatile tools that you can combine quickly to achieve a similar outcome.

Historical Context:

- CISC emerged when assembly programming was common, aiming to simplify the programmer's task.

- As high-level programming languages became more prevalent, the direct reliance on assembly decreased, paving the way for RISC's advantages.

Real-World Examples:

- Many familiar processors, like the x86 CPUs found in most personal computers, are CISC.
- ARM processors (common in smartphones and PDAs), PowerPC, and Sparc are examples of RISC architectures.

Ultimately, the choice between CISC and RISC involves trade-offs, and both have proven successful in different applications.

Difference

This section highlights the key distinctions between CISC (Complex Instruction Set Computer) and RISC (Reduced Instruction Set Computer) architectures, building on our understanding of their fundamental approaches to instruction sets.

Here's a breakdown of their differences:

• Focus:

CISC: Focuses on hardware complexity to handle sophisticated operations.*

RISC: Focuses on software simplicity, with the idea that simpler instructions can be executed more efficiently by the hardware.*

• Registers:

CISC: Requires a larger number of registers. Registers are small, fast storage locations within the CPU used to hold data and instructions that are currently being processed.*

RISC: Requires fewer registers. This is because RISC instructions are simpler and often operate directly on memory, reducing the need to store intermediate results in registers.*

• Operations:

- CISC: Can perform operations that move data directly between memory and registers, or even between memory locations (MEM to MEM).

- RISC: Primarily performs operations that involve registers (REG to REG) or between registers and memory (REG to MEM). Direct memory-to-memory operations are generally not supported.

• Instruction Size:

CISC: Instructions are variable in size. This means some instructions take up more memory space than others.*

RISC: Instructions are fixed in size. This uniformity helps in faster fetching and decoding of instructions.*

• Hardware vs. Software Complexity:

CISC: Uses transistors for more registers and complex circuitry to handle multi-step instructions.*

RISC: Uses transistors for storing complex instructions in software (as sequences of simpler instructions) rather than in hardware.*

- **Code Size:**

*CISC: Results in a larger code size** because complex operations are represented by single, often larger, instructions.

*RISC: Results in a smaller code size** because programs are built from a larger number of simpler instructions, but each instruction itself is typically smaller.

- **Execution Time:**

*CISC: An instruction typically takes more than one clock cycle** to execute due to its complexity.

*RISC: An instruction is designed to be executed in a single clock cycle**. This is a core principle that allows for faster overall processing when pipelining is used effectively.

Instruction and Instruction Types

Beyond the broad categories of RISC and CISC architectures, the specific characteristics of a computer's instruction set provide further ways to classify it. The instruction set acts as the crucial interface between the software written by programmers and the actual hardware of the computer. These instructions tell the hardware precisely what actions to perform under given conditions.

Instructions can vary in several key aspects:

- **Addressing Modes:** How the computer specifies the location of data or instructions to be operated on. Think of this like different ways to give directions to a specific house – you could give a street address, or describe it relative to a landmark.
- **Numbers of Operands:** The quantity of data items (operands) that an instruction can directly work with. Some instructions might work on one piece of data, while others might need two or more.
- **Types of Operations Supported:** The range of actions the CPU can perform, such as arithmetic (add, subtract), logic (AND, OR), or data transfer.
- **Instruction Length:** Whether instructions are of a fixed size or can vary in length. Fixed-length instructions can simplify the fetching process, while variable-length instructions can be more space-efficient.

Word length

Processors can be characterized by their 'word length', which refers to the size of data units they typically operate on. Common word lengths include 4-bit, 8-bit, 16-bit, 32-bit, and so on. In some computer designs, the length of a data word, an instruction, and a memory address are all the same. However, especially in computers designed for smaller word lengths, instructions and addresses might be longer than the basic data word.

Separately from the instruction set, we can also categorize processors by how they execute instructions. A 'single-issue' processor handles one instruction at a time, even if multiple instructions are in different stages of their execution pipeline. In contrast, 'multiple-issue' processors, like 'superscalar' processors, use special logic to identify and execute multiple instructions simultaneously during runtime.

The 'programming model' (or 'programmer model') is the set of registers that are available for programs to use. It's important to note that the CPU has many other registers for its internal operations that are not accessible to programmers.

INSTRUCTION SEQUENCING

Instruction sequencing describes how a computer processes instructions one after another. The processor control circuits use the Program Counter (PC) to fetch and execute instructions in order of increasing memory address. This step-by-step execution is known as straight-line sequencing.

Executing an instruction typically involves a two-phase process:

1. **Instruction Fetch:** The instruction located at the memory address held in the PC is retrieved from memory. This fetched instruction is then stored in the processor's Instruction Register (IR).
2. **Instruction Execute:** The instruction currently in the IR is examined to determine the specific operation that needs to be performed.

Computer instructions can be broadly categorized into four fundamental types:

1. **Data Transfer between Memory and Processor Registers:** Moving data back and forth.
2. **Arithmetic & Logic Operations on Data:** Performing calculations and logical comparisons.
3. **Program Sequencing & Control:** Directing the flow of program execution.
4. **I/O Transfers:** Communicating with input/output devices.

Instructions can also be classified by how they specify the data they operate on:

- **Three-Address Instructions:** These instructions use three locations to specify operands and results. For example, `A + B -> C` (where A, B, and C are memory locations or registers).
- **Two-Address Instructions:** These instructions use two locations, often implying one of the locations serves as both an operand and the destination. For example, `Add contents of R1 to R2 & place sum in R3` (where R1, R2, and R3 are registers).
- **One-Address Instructions:** These instructions use a single address, often implicitly using a special register called an accumulator to hold one of the operands and the result. For example, `Add contents of A to accumulator & store sum back to accumulator`.
- **Zero-Address Instructions:** These instructions do not explicitly mention any addresses. They often rely on a special structure called a push-down stack to store operands and results. The left-hand side of a Register Transfer Notation (RTN) names a location, while the right-hand side denotes a value.